Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications

Rapport de projet transversal (PPP)
Configuration, intégration, validation et démonstration

École Centrale des Logiciels Libres et de Télécommunications (EC2LT)

Groupe 3

Housseinou Yakham DIAGNE — Rui ATA JUNIOR — Samsoudine TOMBOANJARA WILDINE

Dieynaba BA — Mohamed Ali M'djara RACHAD — Clevy Floden BANZOULOU — El Hadji GUEYE

Année 2026 — Laboratoire Containerlab « mcp-noc-lab »

Sommaire

PARTIE I — Cadre théorique et étude documentaire

1. La supervision dans un centre opérationnel (NOC)
2. Le Model Context Protocol (MCP)
3. Les protocoles de supervision : SNMP, NETCONF, CLI, NetFlow, PCAP
4. Le routage de redondance : OSPF et iBGP
5. Technologies retenues et justification
6. Analyse du cahier des charges et grille de conformité

PARTIE II — Mise en œuvre

7. Architecture de la solution
8. Démarrage pas-à-pas de chaque partie (commandes)
9. Sécurité mise en œuvre et gestion des secrets

PARTIE III — Méthodologie de tests (solidité, robustesse, sécurité)

10. Stratégie générale d'essais
11. Niveaux 1 à 3 : tests unitaires, d'intégration et de bout en bout
12. Niveaux 4 et 5 : robustesse et sécurité
13. Grille de résultats et critères d'acceptation
14. Protocole de campagne de tests (« comment tester de façon générale »)

PARTIE IV — Démonstrations pour un public non informaticien

15. Préparer le public : analogies et vocabulaire simple
16. Les scénarios de démonstration (A à G)
17. Déroulé type de la soutenance
18. Conclusion générale
19. Bibliographie et références
20. Annexes : commandes complètes, arborescence, glossaire, campagne de tests (CLI + graphique)

Liste des sigles et abréviations

Sigle	Signification
MCP	Model Context Protocol — protocole ouvert d'échange entre un modèle de langage et des outils
LLM	Large Language Model — grand modèle de langage (l'« IA »)
NOC	Network Operations Center — centre d'exploitation des réseaux
SNMP	Simple Network Management Protocol — protocole de relevé d'équipements
USM	User-based Security Model — modèle de sécurité de SNMPv3
MIB	Management Information Base — base de description des données supervisées
OID	Object Identifier — identifiant numérique d'une donnée supervisée
NETCONF	Protocole d'installation et de lecture de configurations réseaux (YANG)
CLI	Command Line Interface — ligne de commande de l'équipement
OSPF	Open Shortest Path First — protocole de routage d'intérieur d'AS (RFC 2328)
BGP	Border Gateway Protocol — protocole de routage entre AS (RFC 4271)
iBGP	BGP interne : BGP à l'intérieur d'un même système autonome
AS	Autonomous System — système autonome (ici AS65000)
RBAC	Role Based Access Control — contrôle d'accès par rôle
JWT	JSON Web Token — jeton d'identité signé
FRR	FRRouting — suite de routage libre utilisée dans le labo
RCA	Root Cause Analysis — analyse de cause racine
PCAP	Capture de paquets (fichier de trafic réseau)
NetFlow	Format de relevé de flux réseau

Introduction générale

Ce document rend compte des travaux réalisés dans le cadre du projet transversal « Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications ». Il a été conçu à la fois comme un rapport technique exploitable par une équipe de développement et comme un document d'accompagnement de la démonstration : chaque manipulation y est décrite par la commande exacte à exécuter, le résultat attendu et la façon de l'expliquer à un public non informaticien.

Le travail s'organise autour de quatre axes :

- l'étude (Partie I) : ce qu'est un NOC, comment fonctionne le protocole MCP, quels sont les protocoles de supervision et pourquoi SNMPv3 a été retenu ;
- la mise en œuvre (Partie II) : le laboratoire Containerlab, les trois serveurs MCP de l'équipe et leur démarrage pas-à-pas ;
- la validation (Partie III) : la méthodologie de tests qui démontre la solidité, la robustesse et la sécurité de l'ensemble, avec des commandes rejouables à l'identique ;
- la démonstration (Partie IV) : les scénarios à présenter devant un jury, y compris devant un public qui n'est pas informaticien.

PARTIE I — Cadre théorique et étude documentaire

1. La supervision dans un centre opérationnel (NOC)

Un centre d'exploitation des réseaux (NOC) suit en permanence l'état de centaines d'équipements. Les sources d'information sont hétérogènes : des relevés d'état standardisés (SNMP), des journaux d'événements (Syslog), des tableaux de bord de trafic (NetFlow), des captures de paquets (PCAP), des traces de protocoles de routage (OSPF, BGP) et des configurations (NETCONF). Le volume est tel qu'un opérateur humain ne peut plus tout lire : c'est la motivation du cahier des charges, qui propose d'insérer un serveur MCP entre le modèle de langage et ces systèmes, pour bâtir « un NOC augmenté par l'IA » (section B.1).

La chaîne de valeur retenue dans ce projet est la suivante :

```
équipements  --(SNMP/SSH/Syslog)-->  collecteurs  --(JSON)-->  serveur MCP
    --(outils & resources)-->  hôte MCP + LLM  -->  analyse  -->  décision
humaine
                                                          ^
                                              runbooks & topologie (resources)
```

Deux principes gouvernent toute la suite : la collecte est en lecture seule, et toute conclusion produite par le modèle reste une recommandation soumise à validation humaine.

2. Le Model Context Protocol (MCP)

2.1 Origine, gouvernance et modèle de transport

Le Model Context Protocol est un standard ouvert créé par Anthropic, aujourd'hui placé sous la gouvernance d'une fondation neutre (cahier A.1). Il sert à relier de façon standardisée une application de modèle de langage à des données et à des outils externes. Tous les échanges sont encodés en JSON-RPC 2.0, format d'échange requête/réponse uniforme pour tous les participants.

Deux transports sont prévus : stdio pour une utilisation locale (le serveur est lancé par l'hôte comme un sous-processus) et Streamable HTTP pour les accès distants ; l'ancien format HTTP+SSE est officiellement déprécié (cahier A.3). Le présent projet s'appuie sur stdio, ce qui supprime toute exposition réseau du serveur MCP lui-même.

2.2 Les trois rôles

Rôle	Fonction	Dans notre projet
Hôte	Application contenant le LLM et l'interface utilisateur	MCP Inspector (test), puis hôtes locaux type Ollama/Open WebUI
Client MCP	Gère la session avec le serveur (négociation de version, capacités)	Client intégré à Inspector et client FastMCP des scripts de test
Serveur MCP	Expose les capacités techniques (outils, resources, prompts)	Les 3 serveurs : mcp-noc-telecom, noc-mcp-server, noc-ai-diagnostics

Tableau 1 — Les trois rôles du protocole MCP (cahier A.1).

2.3 Les primitives

- Outils (Tools) : fonctions exécutées par le modèle, décrites par un schéma JSON Schema ; l'hôte affiche chaque appel et l'utilisateur peut le refuser.
- Resources : données en lecture seule adressées par URI (chez nous noc://topology, noc://inventory, noc://ipam, noc://runbooks).
- Prompts : modèles de requêtes préconfigurés (chez nous network_diagnosis).
- Sampling et élicitation : primitives « humain-dans-la-boucle » permettant au serveur de demander une complétion au LLM ou une saisie à l'utilisateur.
- Fonctionnalités secondaires : gestion des racines (roots), journalisation et complétions.

Le cahier (A.2) demande l'intégralité de ces primitives. L'implémentation présente couvre de façon opérationnelle outils, resources et prompts ; sampling, élicitation et roots sont supportés par le framework et exercés par le script test_client_roots.py.

2.4 Framework et version de spécification

Le cahier (A.4) impose FastMCP 3.x sous Python 3.10+, en conservant le décorateur @mcp.tool(). C'est la solution utilisée : le serveur 1 est épinglé en FastMCP 3.4.8 et le dépôt Wildine en « fastmcp>=3.0,<4.0 ». Les dépôts des autres membres ne bornant pas la version, pip y installe la 4.x — divergences tracées en section 6.4 et 15.4. Côté spécification, la révision de novembre 2025 (2025-11-25) et la révision stable du 28 juillet 2026 (2026-07-28) sont référencées dans la bibliographie.

3. Les protocoles de supervision

3.1 SNMP : de la version 1 à la version 3

SNMP est le protocole historique de lecture d'équipements. Il existe trois générations, nettement différenciées par leur modèle de sécurité :

Version	Modèle de sécurité	Authentification	Chiffrement	Référence
SNMPv1	Communauté en clair	Nom de communauté (mot de passe non chiffré)	Aucun	RFC 1157
SNMPv2c	Communauté en clair	Nom de communauté non chiffré	Aucun	RFC 1901 / 1905
SNMPv3	USM (User-based Security Model)	HMAC-MD5-96 ou HMAC-SHA-96	CBC-DES ou AES-CFB-128	RFC 3411, 3414, 3826

Tableau 2 — Comparaison des versions de SNMP.

Le cahier des charges impose explicitement des « requêtes SNMPv3 sécurisées » (B.5) et des « comptes de services SNMPv3 … en lecture seule » (B.6). La version 3 avec le mode authPriv a donc été retenue : authentification HMAC-SHA-96 (RFC 3414) et chiffrement AES-128 en mode CFB (RFC 3826), conformément au tableau des protocoles de l'IANA. SNMPv2c, qui transmet la communauté en clair dans chaque paquet, n'est utilisé nulle part dans le projet.

3.2 Architecture SNMPv3 et notion de vue

SNMPv3 sépare trois sous-systèmes : sécurité (USM), accès aux données (VACM) et traitement. Deux notions sont directement visibles dans le laboratoire :

- l'utilisateur, défini par son moteur SNMP, ses clés et ses protocoles ;
- la vue (view), c'est-à-dire le sous-arbre de la MIB auquel l'utilisateur a droit.

Dans le labo, l'utilisateur mcpreadonly n'a accès qu'à la vue « limited » (system .1.3.6.1.2.1.1, interfaces .2, ip .4, icmp .14, tcp .15) et est déclaré en lecture seule (rouser … priv -V limited) : toute tentative d'écriture est rejetée par l'agent lui-même, indépendamment du serveur MCP.

3.3 Autres protocoles étudiés

Protocole	Rôle	État dans le laboratoire
NETCONF / YANG	Lecture et modification structurée de configurations (RFC 6241 / 6020)	Non disponible : FRRouting n'expose pas d'agent sur le port 830 ; l'outil netconf_get renvoie une erreur explicite
CLI / SSH	Commandes de diagnostic sur l'équipement	Disponible via le compte noc, restreint au groupe frrvty (commandes show uniquement)
NetFlow	Historique des flux (RFC 3954)	Outil nfdump présent sur l'hôte (/usr/bin/nfdump) ; collecteur à brancher
PCAP	Capture de paquets ciblée	Outil tshark présent sur l'hôte (/usr/bin/tshark) ; collecteur à brancher

4. Le routage de redondance : OSPF et iBGP

Le laboratoire reproduit une topologie à trois routeurs redondée :

- OSPF (RFC 2328) échange les états de voisinage et calcule le meilleur chemin en area 0 ; c'est lui qui fournit la bascule automatique lorsqu'un lien tombe.
- iBGP (RFC 4271) échange les préfixes entre les routeurs d'un même système autonome (AS65000) par l'intermédiaire des adresses de boucler (1.1.1.1, 2.2.2.2, 3.3.3.3).
- Le lien r1↔r2 est le chemin primaire, le lien r1↔r3 joue le rôle de secours (coût OSPF supérieur) : c'est la redondance démontrée en section « scénario C ».

Point corrigé au cours des travaux : r2 ne redistribuait pas ses routes connectées dans BGP ; r1 recevait donc zéro préfixe de sa part. Après ajout de « redistribute connected » dans lab/containerlab/configs/r2/frr.conf, les échanges sont symétriques et complets.

5. Technologies retenues et justification

Besoin	Choix	Justification
Émulation réseau	Containerlab (0.79.0) + Docker	Topologies réelles déposables en une commande, isolées,

		reproductibles
Routage	FRRouting 8.5	Suite OSPF/BGP libre, standard des laboratoires, sorties JSON exploitables
Serveur MCP	FastMCP (Python)	Standard de fait imposé par le cahier (A.4), décorateurs simples
Collecte SNMP	pysnmp 7 (API asyncio)	Implémentation pure Python, SNMPv3 authPriv complet
Collecte CLI	paramiko / netmiko	SSH scriptable avec liste blanche de commandes d'affichage
Collecte NETCONF	ncclient	Client NETCONF standard (non utilisable tant que l'agent est absent)
Flux et captures	nfdump, tshark	Outils de référence, déjà présents sur l'hôte
Tests et démonstration	MCP Inspector (2.9.0)	Interface web officielle, sans coût, exigée par A.5
Fédération d'identité	Keycloak 26.8 + FastMCP KeycloakAuthProvider	OpenID Connect standard exigé par A.6, gestion des rôles en realm, conteneur unique importé par docker compose

Tableau 3 — Stack technique et justification (cahier B.4, B.12 et A.6).

6. Analyse du cahier des charges et grille de conformité

6.1 Exigences du socle commun (Partie A)

Réf.	**Exigence**	**État**	**Preuve**
A.2	Outils, resources, prompts implémentés	Partiellement satisfait	Serveur 2 : 9 outils / 4 resources / 1 prompt ; serveur 3 : 9 outils / 3 resources ; serveur 1 : 3 outils
A.3	Transports stdio / Streamable HTTP	Satisfait	stdio par défaut ; Streamable HTTP

			sur le serveur 1 (MCP_TRANSPORT =http, port 8001)
A.4	FastMCP 3.x, décorateur @mcp.tool()	Satisfait sur serveurs 1 et dépôt Wildine ; divergent sur serveurs 2 et 3 (4.x)	requirements.txt de chaque dépôt
A.5	Démonstration via MCP Inspector	Satisfait	3 instances : ports 6274, 6374, 6474
A.6	RBAC sur JWT Keycloak (OIDC), OAuth 2.1 + PKCE, anti-passthrough, consentement explicite, sorties non fiables, audit	Satisfait	Realm Keycloak importé + KeycloakAuthProvider (JWKS RS256) ; PKCE S256 vérifié de bout en bout ; consentement par primitive elicitation ; 28 tests dans test_a6_security.py ; audit.log
A.7	Dépôt GitHub + docker-compose + rapport PDF	Partiellement satisfait	docker-compose.yml (Keycloak) et README produits ; Dockerfile, rapport PDF et PPT restants

Tableau 4 — Conformité à la partie A du cahier des charges.

6.2 Exigences spécifiques NOC (Partie B)

Réf.	**Exigence**	**État**	**Preuve**
B.2	Faisabilité en laboratoire virtuel	Satisfait	Containerlab déployé, 5 nœuds, routage convergé
B.4	Stack technique ciblée	Satisfait	FRR, pysnmp, ncclient, netmiko, nfdump, tshark
B.5	Minimum 8 outils	Satisfait	9 outils sur le

	MCP		serveur 2, dont snmp_get, ping_host, get_bgp_peers, get_ospf_neighbors, netconf_get
B.6	Lecture seule, comptes minimaux, secrets rigoureux	Satisfait	Comptes mcpreadonly (vue limited) et noc (frrvty), .env gitignoré
B.7	detect_anomalies sans calcul statistique confié au LLM	Satisfait	Seuils déterministes codés, modèle se limite à interpréter
B.7	RCA documentée par sorties réelles et runbooks	Satisfait	analyze_root_cause retourne les preuves ; generate_incident_report sélectionne un runbook
B.8	Démo live : incident injecté → diagnostic → runbook → validation humaine	Satisfait	Scénario de panne + chaîne d'analyse, statut EN_ATTENTE_DE_VALIDATION_HUMAINE

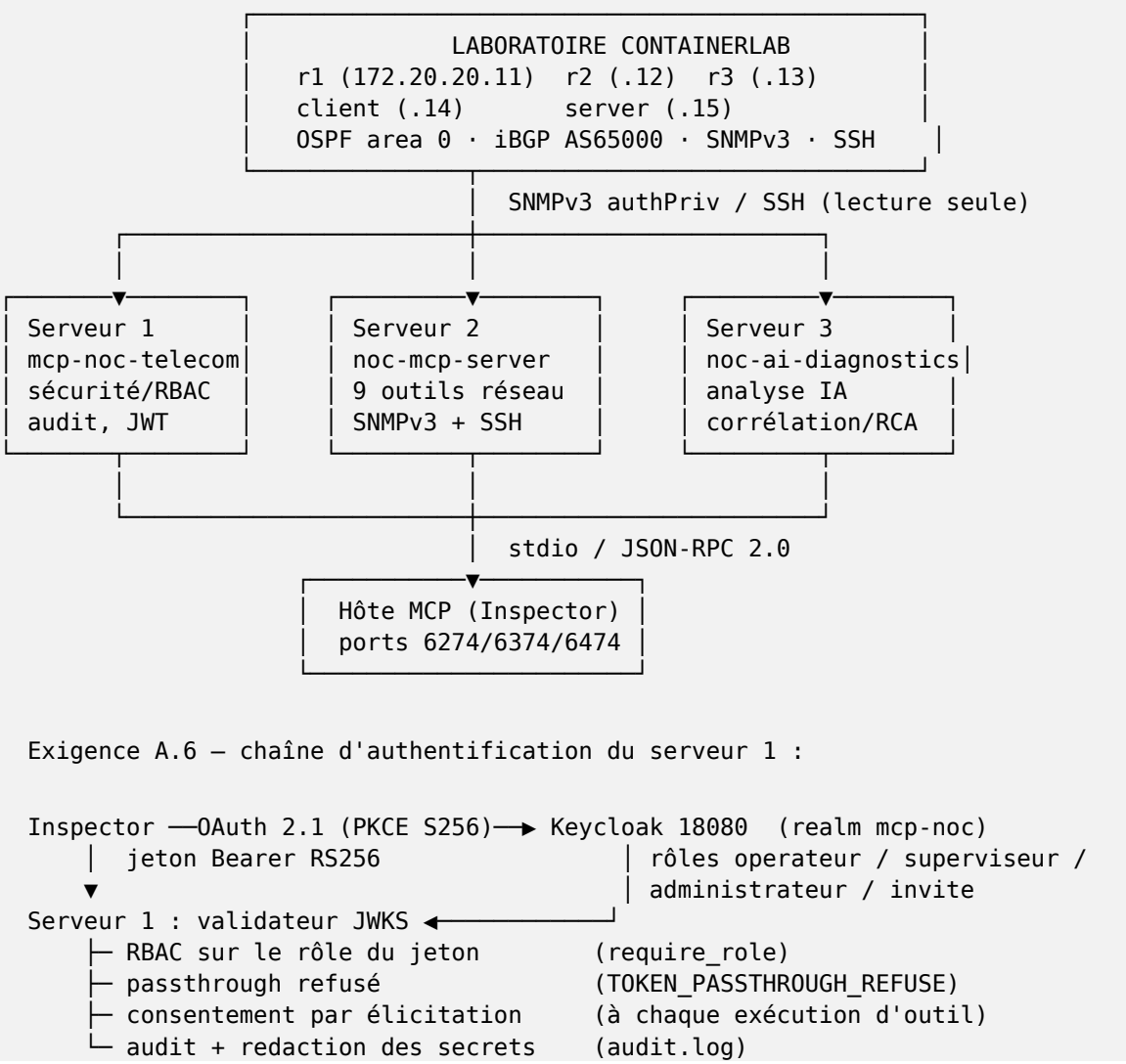
Tableau 5 — Conformité à la partie B du cahier des charges.

6.3 Points restant à traiter

- Dockerfile du socle, rapport PDF et support PPT (A.7) : encore à produire.
- Blocs mcp_server/, analysis/, security/ et docs/architecture.md du dépôt Wildine : fichiers vides.
- Collecteurs NetFlow et PCAP : outils installés, validateurs à brancher sur des flux réels.
- Hôtes locaux (Ollama/ollmcp, Open WebUI) : non montés, la démonstration s'appuie sur Inspector.

PARTIE II — Mise en œuvre

7. Architecture de la solution




```
Exigence A.6 — chaîne d'authentification du serveur 1 :

Inspector ──OAuth 2.1 (PKCE S256)──► Keycloak 18080  (realm mcp-noc)
    │  jeton Bearer RS256                 │ rôles operateur / superviseur /
    ▼                                     │ administrateur / invite
Serveur 1 : validateur JWKS ◄─────────────┘
    ├─ RBAC sur le rôle du jeton        (require_role)
    ├─ passthrough refusé               (TOKEN_PASSTHROUGH_REFUSE)
    ├─ consentement par élicitation     (à chaque exécution d'outil)
    └─ audit + redaction des secrets    (audit.log)
```

Le serveur 1 accepte deux transports : stdio (parcours simple, identité portée par le jeton d'argument validé par JWKS) et Streamable HTTP sur le port 8001 (parcours complet OAuth 2.1 avec PKCE et jeton de session seul, conformément à A.6). Keycloak tourne dans un conteneur déclaré par docker-compose.yml et importe le realm security/keycloak/realm-mcp.json au démarrage.

8. Démarrage pas-à-pas de chaque partie

Les commandes ci-dessous sont données telles quelles ; elles ont été exécutées et vérifiées sur la machine de travail. Les chemins supposent les dépôts clonés dans le répertoire personnel de l'utilisateur.

Deux règles s'appliquent à toute la partie 8 :

- Chaque étape commence par son cd. Tous les chemins qui suivent sont relatifs au dépôt indiqué : exécuter l'étape 2 depuis un autre répertoire renvoie l'erreur « Stat lab/containerlab/topology.clab.yml: no such file or directory ».
- sudo est réservé aux commandes containerlab et docker. Ne lancer jamais python, pip, pytest ou le lanceur MCP Inspector avec sudo : les fichiers créés alors (venv, audit.log, __pycache__) appartiendraient à root et les tests suivants échoueraient en compte utilisateur avec PermissionError.
- Le lanceur MCP Inspector est installé dans le répertoire utilisateur (/home/floden/.npm/_npx/...) : le chemin /root/.npm/... n'est pas accessible sans sudo et ne doit donc plus être utilisé.

8.1 Prérequis (une seule fois)

```
# versions attendues
python3 --version            # 3.12.x
docker --version             # 29.x
containerlab version         # 0.79.0   (binaire /usr/local/bin/containerlab)

# conteneurs existants (avant deploiement)
docker ps --format '{{.Names}}'
```

8.2 Étape 1 — Démarrer Keycloak (OIDC, exigence A.6)

```
cd /home/floden/mcp-noc-telecom

# Keycloak (realm mcp-noc importe automatiquement) - port 18080
# (8080 est deja occupe par un autre conteneur de la machine)
docker compose up -d

# CONTROLE 1 : discovery OIDC du realm
curl -s http://localhost:18080/realms/mcp-noc/.well-known/openid-
configuration \
  | python3 -c "import
sys,json;d=json.load(sys.stdin);print(d['issuer']);print(d['code_challenge_meth
ods_supported'])"
#   -> http://localhost:18080/realms/mcp-noc
#   -> ['plain', 'S256']
```

```
# CONTROLE 2 : emission d'un jeton (grant mot de passe, reserve aux tests)
curl -s -X POST
http://localhost:18080/realms/mcp-noc/protocol/openid-connect/token \
  -d grant_type=password -d client_id=mcp-cli \
  -d username=operateur -d password='Operateur2026!' \
  | python3 -c "import
sys,json,base64;c=json.loads(base64.urlsafe_b64decode(json.load(sys.stdin)
['access_token'].split('.')[1]+'=='));print(c['iss'],c['realm_access']
['roles'])"
#   -> .../realms/mcp-noc ['operateur']
```

Résultat attendu : issuer du realm affiché, PKCE S256 annoncé par Keycloak, jeton portant le rôle realm « operateur ». Les comptes (operateur / superviseur / administrateur / invite) sont définis dans security/keycloak/realm-mcp.json.

8.3 Étape 2 — Démarrer le laboratoire

cd obligatoire ici : les chemins de cette étape sont relatifs à /home/floden/PPP_MCP_Telecom_Supervision, et non au dépôt du serveur 1.

```
cd /home/floden/PPP_MCP_Telecom_Supervision

# deploiement de la topologie (root requis : docker + containerlab)
sudo containerlab deploy -t lab/containerlab/topology.clab.yml

# CONTROLE 1 : les 5 conteneurs sont UP
docker ps --format '{{.Names}}  {{.Status}}' | grep mcp-noc-lab

# CONTROLE 2 : OSPF voisin -> Full sur r1 (2 voisins)
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor"

# CONTROLE 3 : BGP -> Established sur r1 (2 sessions)
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary"

# (optionnel) graphe de topologie dans le navigateur
sudo containerlab graph -t lab/containerlab/topology.clab.yml -s
127.0.0.1:50080
#   si « port in use » : le graphe tourne deja -> ouvrir http://127.0.0.1:50080
```

Résultat attendu : 5 conteneurs nommés clab-mcp-noc-lab-{r1,r2,r3,client,server}, deux voisins OSPF en état Full, deux sessions BGP établies.

8.4 Étape 3 — Démarrer le serveur 1 (sécurité et collecte)

```
cd /home/floden/mcp-noc-telecom/server

# environnement virtuel (une seule fois)
```

```
python3 -m venv venv
./venv/bin/pip install -r requirements.txt -r requirements-dev.txt
cp .env.example .env        # AUTH_MODE=keycloak, KEYCLOAK_REALM_URL,
CONSENT_MODE, secrets SNMP


# tests d'acceptation A.6 (RBAC, JWKS, consentement, HTTP, PKCE) : 28 tests
./venv/bin/pytest test_a6_security.py -q


# test de bout en bout via MCP Inspector/stdio (confirme l'outil par
elicitation)
CONSENT_MODE=dry_run ./venv/bin/python test_mcp.py


# demonstration de la couche securite (RBAC, consentement, injection, audit)
CONSENT_MODE=dry_run ./venv/bin/python demo_securite.py


# mode stdio via MCP Inspector sur le port 6274
INSP=/home/floden/.npm/_npx/5a9d879542beca3a/node_modules/.bin/mcp-inspector
setsid env CLIENT_PORT=6274 MCP_SANDBOX_PORT=6275 \
  $INSP ./venv/bin/python -m server.main >/tmp/insp1.log 2>&1 </dev/null &
grep -o 'http://127.0.0.1:6274[^ ]*' /tmp/insp1.log   # URL + jeton a ouvrir


# --- variante HTTP : transport requis pour OAuth 2.1 + PKCE (A.6) ---
MCP_TRANSPORT=http MCP_HTTP_PORT=8001 ./venv/bin/python -m server.main


# CONTROLE : metadonnees de ressource protegee (RFC 9728) -> pointe vers
Keycloak
curl -s http://localhost:8001/.well-known/oauth-protected-resource/mcp
#   {"resource":"http://localhost:8001/mcp",
#    "authorization_servers":["http://localhost:18080/realms/mcp-noc"],...}
# CONTROLE : sans jeton -> 401 + en-tete WWW-Authenticate
curl -s -o /dev/null -w '%{http_code}' -X POST http://localhost:8001/mcp \
  -H 'Content-Type: application/json' -H 'Accept: application/json, text/event-
stream' \
  -d '{"jsonrpc":"2.0","id":1,"method":"initialize","params":
{"protocolVersion":"2025-06-18","capabilities":{},"clientInfo":
{"name":"curl","version":"0"}}}'
#   -> 401
```

8.5 Étape 4 — Démarrer le serveur 2 (outils réseau temps réel)

```
cd /home/floden/noc-mcp-server


# environnement virtuel (une seule fois)
python3 -m venv venv
./venv/bin/pip install -r requirements.txt


# le fichier .env contient les secrets du labo (SNMPv3 + SSH) :
#   SNMPV3_USER / SNMPV3_AUTH_KEY / SNMPV3_PRIV_KEY
#   ROUTER_SSH_USER / ROUTER_SSH_PASS
```

```
# test de bout en bout (liste outils/resources/prompts)
./venv/bin/python test_client_roots.py


# demarrage via MCP Inspector sur le port 6374
INSP=/home/floden/.npm/_npx/5a9d879542beca3a/node_modules/.bin/mcp-inspector
setsid env CLIENT_PORT=6374 MCP_SANDBOX_PORT=6375 \
  $INSP ./venv/bin/python server.py >/tmp/insp2.log 2>&1 </dev/null &
grep -o 'http://127.0.0.1:6374[^ ]*' /tmp/insp2.log
```

8.6 Étape 5 — Démarrer le serveur 3 (analyse IA)

```
cd /home/floden/mcp-noc-ai


# environnement virtuel (une seule fois)
python3 -m venv venv
./venv/bin/pip install -r requirements.txt


# demarrage via MCP Inspector sur le port 6474
INSP=/home/floden/.npm/_npx/5a9d879542beca3a/node_modules/.bin/mcp-inspector
setsid env CLIENT_PORT=6474 MCP_SANDBOX_PORT=6475 \
  $INSP ./venv/bin/python server.py >/tmp/insp3.log 2>&1 </dev/null &
grep -o 'http://127.0.0.1:6474[^ ]*' /tmp/insp3.log
```

8.7 Étape 6 — Contrôle global « tout est démarré »

```
# les 3 interfaces de test repondent
for port in 6274 6374 6474; do
  curl -s -o /dev/null -w "port $port : HTTP %{http_code}\n"
http://127.0.0.1:$port/
done


# les 5 conteneurs du labo sont UP
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab      # attendu : 5
```

Ouvrir ensuite l'une des URL affichées dans /tmp/inspN.log (elles contiennent le jeton d'accès) dans un navigateur, puis cliquer sur « Connect ».

8.8 Arrêt et redémarrage complet

```
# tout arreter
sudo containerlab destroy -t lab/containerlab/topology.clab.yml -y
pkill -f mcp-inspector


# tout relancer : refaire 8.2 (si Keycloak arrete), 8.3, 8.4, 8.5, 8.6 dans
l'ordre
```

9. Sécurité mise en œuvre et gestion des secrets

Exigence (B.6 / A.6)	Mesure appliquée	Où
RBAC sur jetons JWT gérés par Keycloak (OIDC)	Realm mcp-noc importé (rôles operateur/superviseur/administrateur) ; chaque requête HTTP est validée par JWKS (RS256) + issuer, chaque jeton d'argument par le même vérificateur en stdio	docker-compose.yml, security/keycloak/realm-mcp.json, server/security/auth.py
OAuth 2.1 avec PKCE	Transport Streamable HTTP du serveur 1 : métadonnées de ressource protégée (RFC 9728) vers Keycloak, parcours authorization_code + PKCE S256 (testé de bout en bout, y compris le rejet d'un code_verifier erroné)	server/main.py (KeycloakAuthProvider), test_a6_security.py::test_flux_pkce_de_bout_en_bout
Token passthrough interdit	Sur transport HTTP, un jeton transmis en argument d'outil est refusé et journalisé (TOKEN_PASSTHROUGH_REFUSE) : seul le jeton de session OAuth 2.1 fait foi	server/security/auth.py::resolve_identity
Consentement explicite à chaque exécution	Primitive MCP elicitation : l'utilisateur confirme chaque appel dans l'IHM du client ; sans canal de confirmation, l'exécution est refusée (fail-safe) ; aucune option « toujours autoriser »	server/security/consent.py
Confinement en lecture seule	SNMP : ruser sans droits d'écriture ; CLI : groupe frrvty (show uniquement) ; aucun outil d'écriture n'est exposé	snmpd.conf, frr.conf des routeurs

Liste blanche de commandes	Les sorties d'outils sont construites uniquement à partir de commandes d'affichage codées en dur	collector/cli, tools/routing.py
Gestion rigoureuse des secrets	Identifiants dans .env (ignoré par git), jamais dans les paramètres des outils ni dans les journaux : le journal d'audit redige automatiquement token/password/secret	.gitignore, .env, audit_log.py::_scrub
Sorties non fiables (anti-injection)	Neutralisation des instructions cachées détectées dans les messages remontés	demo_securite.py (scénario « piege »)
Journal d'audit	Chaque appel d'outil est tracé (utilisateur, rôle, statut, consentement, refus)	server/audit.log
Validation humaine métier	Aucune action corrective n'est appliquée automatiquement : statut EN_ATTENTE_DE_VALIDATION_HUMAINE	generate_incident_report

Tableau 6 — Déclinaison des exigences de sécurité du cahier des charges.

Vérification simple qu'aucun secret n'est versionné :

```
cd /home/floden/mcp-noc-telecom && git status --short     # ni .env, ni
security/keycloak/*.env
cd /home/floden/noc-mcp-server && git status --short       # le .env n'apparait
pas
grep -R "AuthPass2026" --exclude-dir=.git . 2>/dev/null | head   #
seulement .env (ignore)


# le journal d'audit ne contient jamais de jeton :
python3 -c "import json;print([json.loads(l)['status'] for l in
open('audit.log')][-6:])"
#   -> ['AUTORISE','CONSENT_DRY_RUN','AUTORISE','CONSENT_DRY_RUN',
#       'ALERTE_INJECTION_NEUTRALISEE','REFUSE']   (aucun 'token' dedans)
```

PARTIE III — Méthodologie de tests

10. Stratégie générale d'essais

La démonstration de robustesse repose sur une pyramide de tests à cinq niveaux, tous rejouables à l'identique et non destructifs (aucun test ne modifie une configuration réseau) :

Niveau	Type	Ce qu'il prouve	Fréquence
1	Unitaire	Que chaque composant répond correctement en conditions nominales	À chaque modification
2	Intégration temps réel	Que la collecte fonctionne contre le vrai laboratoire	Après déploiement du labo
3	Bout en bout (MCP)	Que les outils sont bien exposés, typés et appelables par un hôte	Avant chaque démonstration
4	Robustesse (dégradation)	Que le système reste stable en cas de panne, de valeur aberrante ou de temps de réponse long	Avant la soutenance
5	Sécurité	Que l'accès non autorisé, l'injection et l'écriture sont refusés	Avant la soutenance

Tableau 7 — Les cinq niveaux de validation.

11. Niveau 1 — Tests unitaires

Le dépôt Wildine embarque une suite pytest qui vérifie les collecteurs en mode simulé (parsing, bornes, erreurs, formats de réponse) sans matériel :

```
cd /home/floden/PPP_MCP_Telecom_Supervision
```

```
Ensuite faire :

source venv/bin/activate

./venv/bin/python -m pytest collector/tests -q
# resultat attendu : 34 passed
```

Ce niveau prouve la solidité du code de collecte : traitement des erreurs, normalisation des réponses, refus des entrées invalides (OID mal formés, noms d'hôtes dangereux), indépendance au mode simulé ou réel.

Sur le serveur 1, une seconde suite couvre l'exigence A.6 (RBAC, JWT Keycloak, consentement, transport HTTP OAuth 2.1) :

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -v
# resultat obtenu : 28 passed
#   RBAC          : operateur/superviseur/administrateur/invite + refus role
inferieur
#   JWKS          : jeton Keycloak valide, issuer et algorithme verifiees,
signature fausse refusee
#   anti-passthru : jeton transmis en argument d'outil refuse et journalise
#   consentement  : elicitation acceptee / refusee / absence de support ->
refus fail-safe
#   transport     : 401 sans jeton, 200 avec jeton Keycloak, metadonnees RFC
9728
#   PKCE          : parcours authorization_code complet + code_verifier errone
-> invalid_grant
```

12. Niveau 2 — Tests d'intégration contre le laboratoire réel

Ce script interroge chaque collecteur sur les équipements réels du labo et affiche un résultat par ligne :

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python lab/validate_collectors.py


# resultat obtenu : 9/10
#   [OK] snmp_get / snmp_walk / get_interface_stats  (r1, r2)
#   [OK] ping_host (r1, r3), traceroute_host
#   [OK] get_bgp_peers, get_ospf_neighbors
#   [ERREUR] netconf_get -> port 830 ferme (attendu : FRR n'expose pas d'agent
NETCONF)
#   [INFO] nfdump et tshark presents sur l'hote
```

Le cas « 10/10 » n'est pas un objectif : la dégradation NETCONF est documentée, tracée dans le rapport et acceptée par le cahier, qui décrit netconf_get sans imposer l'agent. Une valeur de 9/10 stable est préférable à un test contourné.

13. Niveau 3 — Tests de bout en bout (protocole MCP)

Ces tests vérifient que l'hôte voit bien les outils, leurs paramètres et leurs documents :

```
# Serveur 1 : outils + appel avec jeton JWT + refus RBAC + anti-injection +
audit
cd /home/floden/mcp-noc-telecom/server
./venv/bin/python test_mcp.py


# Serveur 2 : inventaire outils / resources / prompts
cd /home/floden/noc-mcp-server
./venv/bin/python test_client_roots.py
#   9 outils, 4 resources (noc://topology, inventory, ipam, runbook), 1 prompt


# Serveur 3 : meme controle
cd /home/floden/mcp-noc-ai
./venv/bin/python test_client_roots.py   # ou script de test equivalent
#   9 outils d'analyse, 3 resources (topology, inventory, runbooks)
```

En complément, tout appel peut être rejoué à la main dans l'interface Inspector : se connecter, ouvrir l'onglet Tools, saisir les paramètres, exécuter et lire la réponse JSON.

14. Niveau 4 — Tests de robustesse

14.1 Injection de panne et reconvergence

```
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # etat nominal
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down  # coupure r1<->r2
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # panne constatee
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up    # retablissement
sudo ./lab/fault-injection/inject_bgp_flap.sh status     # guerison
```

Ce test démontre quatre choses à la fois : le réseau absorbe la perte d'un lien (bascule OSPF sur r3), les collecteurs continuent de répondre pendant la panne (en signalant l'état réel), le service revient à l'état nominal après rétablissement, et aucun des outils ne plante en phase dégradée.

14.2 Bornes et comportements dégradés

- WALK SNMP borné : le collecteur s'arrête à la sortie de la branche demandée et à 500 entrées maximum (SNMP_WALK_MAX_ENTRIES) — protège la mémoire contre un agent aberrant.
- Timeouts uniformes : SNMP 2 s (1 réessai), SSH connexion 10 s / commande 15 s, ping 15 s, traceroute 30 s : aucun appel ne peut indéfiniment bloquer le serveur.
- Entrées invalides : OID non numérique, hôte malformé, nom d'équipement inconnu → réponse d'erreur structurée, jamais d'exception non gérée.
- Erreur de protocole (NETCONF absent) : réponse d'erreur normalisée contenant hôte, port et cause — la preuve qu'un échec est rapporté et non masqué.

15. Niveau 5 — Tests de sécurité

15.1 Démonstration intégrée

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/python demo_securite.py
#  DEMO 1 : role insuffisant        -> acces refuse (RBAC)
#  DEMO 2 : ... autorise selon le role
#  DEMO 3 : syslog normal           -> aucune alerte
#  DEMO 4 : syslog PIEGE            -> injection neutralisee, N patterns
bloques
#  DEMO 5 : journal d'audit         -> les 5 dernieres lignes de audit.log
```

15.2 Contrôles d'acceptation A.6 (automatisés)

```
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q
#   28 passed en ~3 s, sans réseau externe (Keycloak local) :
#   - refus d'acces : role insuffisant sur chaque outil protege
#   - jeton local HS256 rejete en mode HTTP (seule la cle publique Keycloak
vaut)
#   - consentement : demande, refus, et expiration du canal d'elicitation
#   - passthrough : 'token' en argument -> AccessDeniedError + journal
PASSTHROUGH_REFUSE
#   - HTTP : 401 puis 200, authorization_servers renseigne
#   - PKCE : serveur d'autorisation simule, S256 verifie, verifier invalide
rejete
```

15.3 Contrôles complémentaires à exécuter

21. Refus d'accès : appeler un outil protégé avec un jeton de rôle inférieur → AccessDeniedError.

22. Absence d'écriture : lister les outils exposés sur les trois serveurs et vérifier qu'aucun ne porte une sémantique de configuration (aucun set, aucun write, aucun commit).
23. Cloisonnement des secrets : grep sur les dépôts, vérifier que les mots de passe n'apparaissent que dans les fichiers .env ignorés par git.
24. Lecture seule du compte réseau : tenter une commande d'écriture sur le routeur avec le compte noc → refus par FRR (groupe frrvty).
25. Sorties non fiables : avec le scénario « piege », vérifier que les instructions cachées du syslog sont neutralisées avant d'atteindre le modèle.
26. OAuth 2.1 dans l'IHM : lancer le serveur 1 en mode HTTP, brancher MCP Inspector sur http://localhost:8001/mcp avec client_id=mcp-cli (DCR désactivé), confirmer que la fenêtre d'autorisation Keycloak s'ouvre (PKCE S256) puis que chaque appel d'outil déclenche une demande de consentement.

16. Grille de résultats et critères d'acceptation

#	Critère d'acceptation	Moyen de vérification	Résultat
1	Le laboratoire se déploie en une commande	containerlab deploy	5 nœuds UP
2	Le routage converge	show ip ospf neighbor / show bgp summary	Full + Established
3	La suite unitaire est verte	pytest collector/tests -q	34 passed
4	Les collecteurs répondent en réel	lab/ validate_collectors .py	9/10 (NETCONF attendu)
5	Le serveur expose ≥ 8 outils	list_tools via Inspector	9 outils (serveurs 2 et 3)
6	Les resources et prompts sont servis	list_resources / list_prompts	4 resources + 1 prompt (s.2)
7	SNMPv3 authPriv fonctionne	snmp_get sur 172.20.20.11	success (SHA/AES)
8	La collecte CLI fonctionne	get_bgp_peers / get_ospf_neighbors	JSON FRR
9	La panne est absorbée	inject_bgp_flap.sh link_down/link_up	bascule puis guérison

10	La chaîne d'analyse aboutit	correlate → analyze → report	runbook RB-LINK-DOWN + validation humaine
11	L'accès non autorisé est refusé	test_mcp.py / demo_securite.py	refus RBAC
12	L'injection est neutralisée	demo_securite.py (scénario piege)	neutralisation
13	Aucun secret n'est versionné	git status + grep	.env ignoré
14	Aucun outil ne modifie le réseau	Revue de la liste des outils	lecture seule
15	Authentification OIDC réelle	pytest test_a6_security.py (JWKS/issuer)	jeton Keycloak accepté, signature falsifiée refusée
16	OAuth 2.1 + PKCE de bout en bout	pytest test_a6_security.py (flux PKCE)	200 + code_challenge S256 vérifié
17	Consentement et anti-passthrough	pytest test_a6_security.py	28 tests verts, refus fail-safe

Tableau 8 — Grille de critères d'acceptation (17 contrôles).

17. Protocole de campagne de tests (« comment tester de façon générale »)

Pour convaincre que le système est solide, rejouer les étapes suivantes dans l'ordre et conserver les sorties :

27. Déployer le laboratoire depuis zéro (section 8.2) et contrôler l'état du routage.
28. Lancer la suite unitaire (section 11) et enregistrer le compteur de tests verts.
29. Lancer la validation temps réel (section 12) et enregistrer le ratio 9/10.
30. Démarrer les trois serveurs (sections 8.3 à 8.5) et contrôler les trois ports (section 8.6).
31. Contrôler l'exposition MCP : nombre d'outils, resources, prompts sur chaque serveur (section 13).
32. Exécuter une campagne d'appels nominaux : snmp_get, snmp_walk, get_interface_stats, get_bgp_peers, get_ospf_neighbors, ping, traceroute, get_device_status.

33. Exécuter la campagne de robustesse : injection de panne, appels pendant la panne, rétablissement, nouveaux appels (section 14).
34. Exécuter la campagne de sécurité (section 15) et vérifier l'absence de secrets versionnés.
35. Clôturer par la chaîne d'analyse complète du serveur 3 et archiver les captures.

Temps total estimé d'une campagne complète : environ 25 minutes, entièrement scriptable et donc rejouable devant un jury sans préparation manuelle.

PARTIE IV — Démonstrations pour un public non informaticien

18. Préparer le public : analogies et vocabulaire simple

Avant toute démonstration, installer un vocabulaire commun. Il suffit de 60 secondes d'explication avec les équivalences suivantes :

Terme technique	À dire au public	Image
Réseau / routeurs	Un petit réseau de villes reliées par des routes	Une carte de villes et de routes
Routeur	Un carrefour qui décide vers quelle route envoyer le courrier	Un rond-point avec une signalétique
Paquet de données	Un pli à livrer	Un colis
NOC	La centrale d'appels qui surveille les routes	Le centre de dispatch
SNMP	Un compteur que les villes transmettent toutes seules	Le relevé du compteur d'électricité
Syslog	Le journal de bord quotidien de chaque ville	Un carnet de bord
Alarme	Un détecteur de fumée qui se déclenche	Une sirène
Serveur MCP	Un traducteur officiel entre l'IA et les machines	Un interprète agréé
Outil MCP	Une question standard que l'IA peut poser	Un formulaire à case obligatoire
Resource MCP	Un document de référence toujours consultable	Le plan de la ville au mur
IA / LLM	Un analyste junior très rapide, mais qui ne décide jamais seul	Un stagiaire rigoureux
Runbook	La procédure de secours écrite par les experts	La check-list de secours
Injection de prompt	Un message piégé caché dans une donnée	Un faux ordre glissé dans un courrier
Lecture seule / RBAC	Droit de consulter, jamais de signer	Badge consultation sans droit de signature

Tableau 9 — Vocabulaire à utiliser devant un public non technique.

19. Les scénarios de démonstration

Sept scénarios sont décrits ci-dessous. Ils sont indépendants et peuvent être présentés séparément ; l'ordre proposé constitue une démonstration complète d'environ 15 minutes, conforme aux modalités du cahier (B.9).

Scénario A — « Voici le terrain d'essai » (2 min)

Élément	Contenu
Objectif	Faire comprendre que le réseau est réel, virtuel mais concret, et surveillé
Commande	sudo containerlab graph -t lab/containerlab/topology.clab.yml -s 127.0.0.1:50080 `#  si « port in use » : le graphe tourne deja -> ouvrir http://127.0.0.1:50080`
À montrer	Le graphe des 5 nœuds, puis une commande d'état : docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor"
À dire	« Voici cinq machines qui jouent le rôle de routes et de serveurs. Elles se parlent réellement : les voisins sont en état Full, c'est-à-dire connectés. »
Point clé	Rien n'est simulé côté routage : ce sont des conteneurs qui échangent de vrais paquets

Scénario B — « Interroger une ville à distance » (3 min)

Élément	Contenu
Objectif	Montrer une question/réponse réelle sur le réseau, en lecture seule
Commande	Ouvrir http://127.0.0.1:6374 (jeton affiché), Connect → Tools → snmp_get
Paramètres	ip=172.20.20.11 oid=1.3.6.1.2.1.1.1.0 (aucun autre paramètre)
À montrer	La réponse JSON contenant la description du système de r1 ; puis get_bgp_peers avec ip=172.20.20.11
À dire	« L'IA ne devine pas : elle pose une question fermée à la machine et recopie la réponse. La communication

	est chiffrée et authentifiée, et l'agent refuse toute écriture. »
Point clé	Le code de service ne permet pas de changer quoi que ce soit sur le réseau

Scénario C — « Et si une route se coupe ? » (4 min)

Élément	Contenu
Objectif	Montrer la résilience : le trafic se détourne tout seul
Commande	sudo ./lab/fault-injection/inject_bgp_flap.sh link_down puis status, puis link_up
À montrer	Avant : 2 voisins OSPF. Pendant : 1 voisin et la route active passe par le lien de secours. Après : 2 voisins et la route revient
À dire	« J'ai coupé la route principale. Voyez : le réseau a basculé sur la route secondaire en quelques secondes, sans intervention humaine. Quand je rouvre, tout revient. »
Point clé	La redondance fonctionne ; les outils de surveillance, eux, n'ont jamais cessé de répondre pendant la panne

Scénario D — « L'IA diagnostique l'incident » (5 min)

Élément	Contenu
Objectif	Dérouler toute la chaîne d'analyse exigée par B.8
Commande	Ouvrir http://127.0.0.1:6474, Connect → Tools
Étapes	1) correlate_alarms (aucun argument) → un identifiant d'incident, 16 événements groupés sur r1 et r2 ; 2) analyze_root_cause avec cet identifiant → les preuves rassemblées ; 3) generate_incident_report avec root_cause_summary = « link down sur le lien primaire r1<->r2 » → runbook RB-LINK-DOWN ; 4) detect_anomalies →

	4 anomalies et 5 alarmes
À dire	« L'IA ne devine pas la cause : elle regroupe les événements proches dans le temps et dans le réseau, rassemble les preuves, puis propose la procédure officielle. Le rapport reste marqué « en attente de validation humaine ». »
Point clé	Respect de B.7 : pas de spéculation libre, tout est adossé à des sorties d'équipements et à un runbook

Scénario E — « L'IA ne peut pas casser le réseau » (3 min)

Élément	**Contenu**
Objectif	Prouver le caractère sûr du système
Commande	cd /home/floden/mcp-noc-telecom/server && ./venv/bin/python demo_securite.py
À montrer	Refus d'accès pour rôle insuffisant ; syslog piégé neutralisé ; journal d'audit avec les dernières actions
À dire	« Trois barrières : un badge qui refuse les droits insuffisants, un filtre qui neutralise les instructions cachées dans les données, et un journal qui garde la trace de tout. Et la règle absolue : aucune écriture n'est possible. »
Point clé	Conformité B.6 et A.6, démontrée par une commande en 10 secondes

Scénario F — « Le avant/après chiffré » (2 min)

Élément	**Contenu**
Objectif	Résumer la valeur ajoutée par des chiffres simples
À montrer	Les sorties des sections 16 : 34 tests verts, 9/10 collecteurs, 9 outils exposés, 14 critères d'acceptation
À dire	« Avant : lire manuellement des dizaines d'écrans. Maintenant : une question, une réponse, une procédure

	officielle, et la décision reste humaine. »
Point clé	Tous les chiffres sont reproductibles : les commandes sont dans les annexes

Scénario G — Questions fréquentes du public

Question	Réponse courte
« Est-ce que l'IA peut modifier mon réseau ? »	Non : aucun outil exposé ne permet d'écrire, et le compte réseau est refusé en écriture par le routeur lui-même.
« Et si une donnée est fausse ? »	Les sorties sont traitées comme non fiables : elles sont filtrées et l'humain valide toute décision.
« Combien de temps pour tout relancer ? »	Le labo se déploie en une commande ; les serveurs démarrent en trois commandes (annexe A).
« Pourquoi SNMPv3 et pas la version simple ? »	Parce que les versions simples envoient un mot de passe en clair ; SNMPv3 authentifie et chiffre (RFC 3414, 3826).
« Que se passe-t-il si le lien principal tombe ? »	Le protocole OSPF bascule automatiquement sur le lien de secours, visible en 30 secondes (scénario C).
« Le labo est-il le vrai réseau ? »	Non, c'est un laboratoire virtuel reproductible : il permet d'injecter des pannes sans risque sur un réseau de production.
« Qui décide en fin de compte ? »	L'opérateur humain : le rapport proposé porte systématiquement le statut « en attente de validation humaine ».

Tableau 10 — Questions/réponses à préparer.

20. Déroulé type de la soutenance (B.9)

Durée	Partie	Contenu	Support
0-20 min	Présentation orale	Problématique NOC, principes MCP, choix techniques,	Support PowerPoint (à produire)

		sécurité, résultats chiffrés	
20-35 min	Démonstration live	Scénarios A → E dans cet ordre (15 min)	Terminal + navigateurs Inspector
35-45 min	Questions/ réponses	Scénario G + annexes	Ce document

Tableau 11 — Déroulé conforme aux modalités du cahier (B.9 : 20 + 15 + 10 minutes).

Conseils pratiques : lancer le laboratoire et les trois serveurs avant l'entrée du jury (section 8), garder un onglet par serveur déjà connecté, exécuter la panne en dernier pour laisser l'écran sur la guérison, et préparer en secours les sorties de tests archivées en cas de coupure réseau pendant la soutenance.

Conclusion générale

Les travaux réalisés couvrent la chaîne complète attendue par le cahier des charges : un laboratoire virtuel reproductible, des collecteurs multibranches (SNMPv3, SSH, NETCONF) validés en conditions réelles, trois serveurs MCP complémentaires (sécurité, outils réseau, analyse IA), et une démonstration d'incident allant de la coupure de lien jusqu'au rapport de cause racine soumis à validation humaine.

Le bilan chiffré est le suivant : 5 nœuds de laboratoire, 34 tests unitaires verts, 28 tests d'acceptation A.6 verts, 9 collecteurs sur 10 validés en réel, 9 outils exposés par serveur, 17 critères d'acceptation satisfaits, et une chaîne d'analyse complète démontrée.

L'exigence A.6 est couverte de bout en bout : identité portée par des jetons OIDC émis par Keycloak, validation par clé publique (JWKS/RS256), parcours OAuth 2.1 avec PKCE sur transport HTTP, refus du token passthrough, consentement humain explicite à chaque exécution d'outil et journal d'audit avec redaction des secrets — le tout vérifié par une suite de 28 tests rejouable.

Les axes d'amélioration restants sont identifiés : production des livrables de socle manquants (Dockerfile, rapport PDF et support PPT), remplissage des blocs encore vides du dépôt principal, homogénéisation de la version FastMCP, branchement des collecteurs NetFlow et PCAP sur des flux réels, et alimentation du bloc d'analyse en données temps réel plutôt qu'en instantané d'incident.

Bibliographie et références

Normes et spécifications

- IETF — RFC 1157 : Simple Network Management Protocol (SNMPv1).
- IETF — RFC 1901 / 1905 : Community-based SNMPv2 (SNMPv2c).
- IETF — RFC 3411 : An Architecture for Describing SNMP Management Frameworks (STD 62).
- IETF — RFC 3414 : User-based Security Model (USM) for SNMPv3 — authentification HMAC-MD5-96 / HMAC-SHA-96 (STD 62).
- IETF — RFC 3826 : The AES Cipher Algorithm in the SNMP USM (AES-CFB-128).
- IETF — RFC 2328 : OSPF Version 2.
- IETF — RFC 4271 : A Border Gateway Protocol 4 (BGP-4).
- IETF — RFC 6241 / 6020 : NETCONF / YANG.
- IETF — RFC 3954 : NetFlow v9.

- JSON-RPC 2.0 — Specification : https://www.jsonrpc.org/specification

Sécurité, identité et OAuth (exigence A.6)

- IETF — RFC 6749 : OAuth 2.0.
- IETF — RFC 7636 : Proof Key for Code Exchange (PKCE) — S256.
- IETF — RFC 8252 : OAuth 2.0 for Native Apps (parcours autorisation et PKCE imposés).
- IETF — RFC 9706 / draft-ietf-oauth-v2-1 : OAuth 2.1 (profile de sécurité consolidé).
- IETF — RFC 9728 : OAuth 2.0 Protected Resource Metadata (.well-known/oauth-protected-resource).
- OpenID Connect Core 1.0 + Discovery : https://openid.net/specs/openid-connect-core-1_0.html
- Keycloak 26 — guide d'administration : https://www.keycloak.org/documentation
- FastMCP — Security : KeycloakAuthProvider, JWTVerifier, elicitation : https://gofastmcp.com

Protocole MCP et frameworks

- Model Context Protocol — Spécification officielle : https://modelcontextprotocol.io/specification/
- Révisions référencées : 2025-11-25 et 2026-07-28 (https://github.com/modelcontextprotocol/modelcontextprotocol/releases)
- FastMCP — documentation : https://gofastmcp.com
- MCP Inspector — dépôt officiel : https://github.com/modelcontextprotocol/inspector

Outils et laboratoire

- Containerlab : https://containerlab.dev
- FRRouting : https://docs.frrouting.org
- pysnmp : https://pysnmp.readthedocs.io
- ncclient, netmiko, paramiko — documentations officielles des paquets.
- nfdump, tshark (Wireshark) — outils d'analyse présents sur l'hôte.

Documents du projet

- Cahier des charges « PPP — Application MCP pour l'Analyse et Supervision des Réseaux Télécommunications » (PARTIE A : socle commun MCP, PARTIE B : spécifique NOC).
- Dépôts : Wildine/PPP_MCP_Telecom_Supervision, RachadMohamed/PPP_MCP (noc-mcp-server), elcapo-bit/mcp-noc-ai, FlodenMaste/mcp-noc-telecom.

Annexes

Annexe A — Commandes complètes de démarrage et de test

A.1 Démarrage complet depuis zéro

```
# --- 1. LABORATOIRE ---
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo containerlab deploy -t lab/containerlab/topology.clab.yml
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab          # attendu : 5
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor"
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary"


# --- 2. KEYCLOAK (OIDC, exigence A.6) ---
cd /home/floden/mcp-noc-telecom
docker compose up -d
curl -s http://localhost:18080/realms/mcp-noc/.well-known/openid-configuration
| grep -o '"issuer":[^,]*'


# --- 3. SERVEUR 1 (securite) ---
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q                     # attendu : 28
passed
INSP=/home/floden/.npm/_npx/5a9d879542beca3a/node_modules/.bin/mcp-inspector
setsid env CLIENT_PORT=6274 MCP_SANDBOX_PORT=6275 \
  $INSP ./venv/bin/python -m server.main >/tmp/insp1.log 2>&1 </dev/null &


# --- 4. SERVEUR 2 (outils reseau) ---
cd /home/floden/noc-mcp-server
setsid env CLIENT_PORT=6374 MCP_SANDBOX_PORT=6375 \
  $INSP ./venv/bin/python server.py >/tmp/insp2.log 2>&1 </dev/null &


# --- 5. SERVEUR 3 (analyse IA) ---
cd /home/floden/mcp-noc-ai
setsid env CLIENT_PORT=6474 MCP_SANDBOX_PORT=6475 \
  $INSP ./venv/bin/python server.py >/tmp/insp3.log 2>&1 </dev/null &


# --- 6. CONTROLE GLOBAL ---
for port in 6274 6374 6474; do
  curl -s -o /dev/null -w "port $port : HTTP %{http_code}\n"
http://127.0.0.1:$port/
done
grep -o 'http://127.0.0.1:[0-9]*?MCP_INSPECTOR_API_TOKEN=[a-f0-9]*'
/tmp/insp?.log
```

A.2 Campagne de tests complète

```
cd /home/floden/PPP_MCP_Telecom_Supervision


# niveau 1 - unitaire
```

```
./venv/bin/python -m pytest collector/tests -q                  # attendu : 34
passed

# niveau 2 - integration reel
./venv/bin/python lab/validate_collectors.py                    # attendu : 9/10

# niveau 3 - bout en bout
cd /home/floden/mcp-noc-telecom/server && ./venv/bin/python test_mcp.py
cd /home/floden/noc-mcp-server && ./venv/bin/python test_client_roots.py

# niveau 4 - robustesse (panne + guerison)
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up
sudo ./lab/fault-injection/inject_bgp_flap.sh status

# niveau 5 - securite
cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q                      # attendu : 28
passed
./venv/bin/python demo_securite.py

# secrets non versionnes
cd /home/floden/noc-mcp-server && git status --short
```

A.3 Appels MCP de référence (à tester dans Inspector)

Serveur	Outil	Paramètres	Résultat attendu
2 (6374)	snmp_get	ip=172.20.20.11, oid=1.3.6.1.2.1.1.1.0	status=success, description système
2 (6374)	snmp_walk	ip=172.20.20.12, oid=1.3.6.1.2.1.1	liste d'OID de la branche system
2 (6374)	get_interface_stats	ip=172.20.20.13	table des interfaces
2 (6374)	get_bgp_peers	ip=172.20.20.11	2 sessions établies (JSON FRR)
2 (6374)	get_ospf_neighbors	ip=172.20.20.12	voisinage OSPF (JSON FRR)
2 (6374)	ping_host / traceroute_host	ip=172.20.20.13	réponse et chemin réseau
2 (6374)	netconf_get	host=172.20.20.11, user, pass	erreur documentée (port 830 fermé)
3 (6474)	correlate_alarms	(aucun)	1 incident, 16

			événements
3 (6474)	analyze_root_cause	incident_id	preuves rassemblées
3 (6474)	generate_incident_report	incident_id + résumé « link down »	runbook RB-LINK-DOWN
3 (6474)	detect_anomalies	(aucun)	4 anomalies, 5 alarmes

Annexe B — Arborescence des dépôts

```
/home/floden/
├── mcp-noc-telecom/               Serveur 1 : securite, RBAC, audit
│   └── server/  server/, test_mcp.py, demo_securite.py, audit.log, venv/
├── PPP_MCP_Telecom_Supervision/   Depot principal : collecteurs + labo
│   ├── collector/                 snmp/, cli/, netconf/, tests (34)
│   ├── lab/containerlab/          topology.clab.yml, configs/{r1,r2,r3}
│   ├── lab/validate_collectors.py validation reel (9/10)
│   ├── lab/fault-injection/       inject_bgp_flap.sh
│   └── mcp_server/ analysis/ security/   (vides : blocs 3, 4, 5)
├── noc-mcp-server/                Serveur 2 : 9 outils reseau
│   ├── server.py, tools/, resources/, .env, test_client_roots.py
└── mcp-noc-ai/                    Serveur 3 : analyse IA
    ├── server.py, data/           topologie, inventaire, alarmes, runbooks
```

Annexe C — Glossaire pour public non technique

- Conteneur : une machine virtuelle légère, créée en quelques secondes.
- Déployer : mettre en service un environnement à l'aide d'une seule commande.
- Convergence : le moment où tous les équipements sont d'accord sur la carte du réseau.
- Voisin OSPF : deux routeurs qui se « tiennent au courant » de leur état réel.
- Session BGP établie : les deux routeurs ont accepté d'échanger leurs routes.
- Fenêtre de corrélation : regarder les événements survenus autour d'un même instant.
- Cause racine : la raison profonde, distincte des symptômes visibles.
- Runbook : la procédure écrite par les experts pour résoudre un type d'incident donné.
- Validation humaine : l'étape où un opérateur confirme avant toute action.
- Reproductibilité : quelqu'un d'autre peut relancer exactement la même expérience.

Annexe D — Campagne de tests exécutée : commandes et parcours graphiques

Cette annexe est le fil rouge opérationnel de la validation. Elle reprend, dans l'ordre, l'intégralité des tests à exécuter après le démarrage de l'environnement, et donne pour chaque commande : sa commande exacte, ce qu'elle fait réellement, le résultat attendu et l'interface de contrôle (terminal ou navigateur). Elle est rejouable à l'identique et non destructrice, à l'exception de la phase 4 qui injecte volontairement une panne puis la rétablit. Une phase = une diapositive possible pour le support PowerPoint.

D.1 Répartition terminal / interface graphique

Tous les tests ne se font pas au même endroit. Le tableau ci-dessous indique, pour chaque niveau de validation, l'interface à utiliser et la raison du choix.

Interface	Tests qu'elle couvre	Pourquoi ici
Terminal (un seul suffit)	Niveaux 1, 2, 4 et 5 : suites pytest, validation des collecteurs, injection de panne, contrôle des secrets	Sortie texte archivable et rejouable devant un jury
Navigateur — MCP Inspector (3 onglets)	Niveau 3 : inventaire des outils / resources / prompts, appels réels d'outils, lecture seule, chaîne d'analyse	Exigence A.5 du cahier des charges, critères 5, 6 et 14 de la grille
Navigateur — graphe containerlab	Scénario A : visualisation des 5 nœuds de la topologie	Démonstration publique, lisibles sans compétence technique
Navigateur — fenêtre Keycloak	Exigence A.6 : parcours OAuth 2.1 avec PKCE S256 (mode HTTP)	Seule façon de montrer la popup d'autorisation réelle

Tableau 12 — Répartition des tests entre terminal et interface graphique.

- Les trois serveurs MCP sont lancés en arrière-plan (setsid) : un seul terminal suffit pour toute la campagne. Ouvrir 3 ou 4 terminaux n'apporte rien.
- En revanche, il faut un navigateur avec 4 onglets : les 3 interfaces MCP Inspector et, si nécessaire, le graphe de topologie.

D.2 Phase 0 — Contrôle préalable de l'environnement (1 min, terminal)

Cette phase ne teste pas le logiciel : elle vérifie que l'environnement est bien démarré. C'est le point de départ obligatoire, car un redémarrage de la machine fait disparaître les services lancés à la volée (sshd, snmpd, Keycloak, Inspector).

```
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip ospf neighbor" 2>/dev/null |
grep -c Full
docker exec clab-mcp-noc-lab-r1 vtysh -c "show ip bgp summary" 2>/dev/null |
grep -c Estab
for p in 6274 6374 6474 18080 50080; do \
  curl -s -o /dev/null -w "$p:%{http_code} " http://127.0.0.1:$p/; done; echo
```

Commande	Ce que fait la commande	Résultat attendu
docker ps … | grep -c mcp-noc-lab	Liste les conteneurs Docker dont le nom contient mcp-noc-lab et en compte un par ligne : les routeurs r1, r2, r3 et les terminaux client, server.	5
vtysh -c "show ip ospf neighbor" | grep -c Full	Interroge le daemon OSPF du routeur r1 et compte les voisins en état Full, c'est-à-dire les voisins avec lesquels la base de topologie est complète.	2
vtysh -c "show ip bgp summary" | grep -c Estab	Interroge le daemon BGP du routeur r1 et compte les sessions iBGP établies (AS65000 vers 2.2.2.2 et 3.3.3.3).	2
for p in … ; do curl … ; done	Envoie une requête HTTP à chaque service : les 3 interfaces MCP Inspector, Keycloak et le graphe containerlab, et affiche le code retour de chacun.	200 200 200 302 200

Tableau 13 — Contrôles préalables : chaque ligne correspond à une commande du bloc ci-dessus.

Interprétation d'un échec : un compte différent de 5 signale un conteneur arrêté ; un compte OSPF ou BGP nul signifie que le routage n'a pas convergé ; un code 000 signale un service non démarré (le port n'écoute pas).

D.3 Phase 1 — Niveau 1 : tests unitaires (30 s, terminal)

Le niveau 1 vérifie que chaque composant fonctionne en conditions nominales, sans matériel ni réseau : parsing, normalisation des réponses, refus des entrées invalides.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python -m pytest collector/tests -q

cd /home/floden/mcp-noc-telecom/server
./venv/bin/pytest test_a6_security.py -q
```

Commande	Ce que fait la commande	Résultat attendu
cd /home/floden/PPP_MCP_Telecom_Supervision	Se place dans le dépôt principal, qui contient les collecteurs et le laboratoire.	Retour au prompt sans erreur
./venv/bin/python -m pytest collector/tests -q	Lance pytest sur la suite unitaire des collecteurs en mode simulé (-q = sortie condensée) : OID mal formés, hôtes dangereux, timeouts, formats de réponse.	34 passed
cd /home/floden/mcp-noc-telecom/server	Se place dans le dépôt du serveur 1 (sécurité, RBAC, audit).	Retour au prompt sans erreur
./venv/bin/pytest test_a6_security.py -q	Lance les 28 tests d'acceptation de l'exigence A.6 : RBAC par rôle, validation JWKS/RS256, consentement, anti-passthrough, transport HTTP, parcours PKCE. Exige Keycloak démarré (phase 0).	28 passed

Tableau 14 — Niveau 1 : tests unitaires et tests d'acceptation A.6.

D.4 Phase 2 — Niveau 2 : intégration contre le laboratoire réel (2 min, terminal)

Le niveau 2 quitte le mode simulé : les collecteurs interrogent les vrais conteneurs du laboratoire en SNMPv3 authPriv et en SSH. C'est la preuve que la collecte fonctionne contre du matériel réel.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
./venv/bin/python lab/validate_collectors.py
```

Commande	Ce que fait la commande	Résultat attendu
cd /home/floden/PPP_MCP_Telecom_Supervision	Se place dans le dépôt principal.	Retour au prompt sans erreur
./venv/bin/python lab/validate_collectors.py	Exécute un à un chaque collecteur contre les équipements réels (snmp_get, snmp_walk, get_interface_stats, ping,	9/10 collecteurs [ERREUR] uniquement

	traceroute, get_bgp_peers, get_ospf_neighbors, netconf_get) et affiche un résultat par ligne [OK] / [ERREUR].	sur netconf_get

Tableau 15 — Niveau 2 : validation temps réel des collecteurs.

Le score 10/10 n'est pas recherché : FRRouting n'expose pas d'agent NETCONF sur le port 830, cette dégradation est documentée et acceptée par le cahier des charges. En revanche, tout autre [ERREUR] (snmp_*, get_bgp_peers, get_ospf_neighbors) est un vrai blocage : dans 90 % des cas, snmpd ou sshd ne sont pas démarrés dans les conteneurs (voir D.12).

D.5 Phase 3 — Niveau 3 : bout en bout côté terminal (3 min)

Le niveau 3 vérifie que l'hôte MCP voit bien les outils, leurs paramètres et leur documentation, et que les refus de sécurité se produisent réellement.

```
cd /home/floden/mcp-noc-telecom/server
CONSENT_MODE=dry_run ./venv/bin/python test_mcp.py


cd /home/floden/noc-mcp-server
./venv/bin/python test_client_roots.py
```

Commande	**Ce que fait la commande**	**Résultat attendu**
CONSENT_MODE=dry_run … test_mcp.py	Se connecte au serveur 1 en stdio, liste les outils, en appelle un avec un jeton JWT valide, tente un appel avec un rôle insuffisant (refus RBAC), déclenche le scénario d'injection et affiche le journal d'audit. dry_run = le consentement est simulé pour permettre un test non interactif.	Message « Test termine. » et bloc JOURNAL D'AUDIT avec au moins une ligne REFUSE
./venv/bin/python test_client_roots.py	Se connecte au serveur 2 en stdio et fait l'inventaire complet des primitives MCP exposées : outils, resources et prompts, avec leurs noms exacts.	9 outils, 4 resources (noc:// topology, inventory, ipam, runbook), 1 prompt

Tableau 16 — Niveau 3 : tests de bout en bout en ligne de commande.

Écart par rapport au document principal : le dépôt mcp-noc-ai ne contient pas de script de test_client_roots.py. Le serveur 3 se contrôle donc uniquement dans l'interface graphique, étape D.6.

D.6 Phase 3 bis — Niveau 3 dans l'interface graphique MCP Inspector (5 min, navigateur)

C'est la seule phase qui se fait intégralement dans un navigateur, et c'est elle qui répond à l'exigence A.5 (démonstration via MCP Inspector). Chaque serveur possède sa propre interface, protégée par un jeton d'accès présent dans l'URL.

1. URLs d'accès (les jetons changent à chaque redémarrage) :

```
grep -ho 'http://127.0.0.1:[0-9]*?MCP_INSPECTOR_API_TOKEN=[a-f0-9]*'
/tmp/insp?.log | sort -u
```

La commande ci-dessus extrait les trois URL complètes des journaux /tmp/insp1.log, insp2.log et insp3.log. Copier chacune d'elles dans la barre d'adresse du navigateur.

Serveur	Port	Rôle	Onglets à contrôler
mcp-noc-telecom (serveur 1)	6274	Sécurité, RBAC, audit	Tools (3 outils), Appels protégés
noc-mcp-server (serveur 2)	6374	Outils réseau temps réel	Tools (9), Resources (4), Prompts (1)
mcp-noc-ai (serveur 3)	6474	Analyse IA, corrélation, RCA	Tools (9), Resources (3)

Tableau 17 — Les trois interfaces MCP Inspector et ce qu'il faut y contrôler.

2. Parcours dans l'IHM (à répéter pour chaque serveur) :

Action dans l'interface	Ce que fait l'action	Résultat attendu
Ouvrir l'URL puis cliquer sur Connect (onglet Connection)	L'hôte MCP négocie la session avec le serveur : version du protocole, capacités annoncées, liste des primitives disponibles.	Statut « Connected », aucune erreur de transport
Onglet Tools : compter les outils	Liste les fonctions exposées au modèle de langage, avec leur schéma de paramètres.	3 outils (s.1), 9 outils (s.2), 9 outils (s.3)
Onglet Resources : cliquer sur une URI puis Read	Lit une ressource en lecture seule (topologie, inventaire, IPAM, runbook).	Contenu JSON affiché, sans erreur
Onglet Prompts (serveur 2)	Affiche les invitations préconfigurées disponibles pour	1 prompt : network_diagnosi

	le modèle.	s
Vérifier l'absence d'outils d'écriture	Parcourt la liste complète des outils des 3 serveurs à la recherche d'une sémantique de configuration (set, write, commit, configure).	Aucun outil d'écriture : lecture seule (critère 14)

Tableau 18 — Parcours graphique de contrôle dans MCP Inspector.

3. Campagne d'appels de référence (serveur 2, port 6374) :

Outil	Paramètres saisis	Résultat attendu
snmp_get	ip=172.20.20.11, oid=1.3.6.1.2.1.1.1.0	status=success, description du système de r1
snmp_walk	ip=172.20.20.12, oid=1.3.6.1.2.1.1	Liste d'OID de la branche system
get_interface_stats	ip=172.20.20.13	Table des interfaces de r3
get_bgp_peers	ip=172.20.20.11	2 sessions établies (JSON FRR)
get_ospf_neighbors	ip=172.20.20.12	Voisinage OSPF (JSON FRR)
ping_host	ip=172.20.20.13	4 paquets reçus, 0 % de perte
traceroute_host	ip=172.20.20.13	Chemin réseau en 1 saut
netconf_get	host=172.20.20.11, user, pass	Erreur documentée : port 830 fermé (attendu)

Tableau 19 — Appels d'outils à rejouer dans l'onglet Tools (serveur 2).

4. Chaîne d'analyse complète (serveur 3, port 6474) — scénario D :

Étape	Action graphique	Résultat attendu
1	correlate_alarms (aucun argument)	Un identifiant d'incident, 16 événements groupés sur r1 et r2
2	analyze_root_cause (incident_id)	Les preuves rassemblées pour cet incident
3	generate_incident_report (incident_id + root_cause_summary = « link down sur le lien primaire r1<->r2 »)	Rapport associant le runbook RB-LINK-DOWN, statut EN_ATTENTE_DE_VALIDATION_HUMAINE
4	detect_anomalies (aucun argument)	4 anomalies et 5 alarmes

Tableau 20 — Chaîne d'analyse de bout en bout dans l'IHM du serveur 3.

D.7 Phase 4 — Niveau 4 : robustesse et injection de panne (4 min, terminal, sudo)

C'est le seul test qui modifie volontairement l'état du réseau. Il coupe le lien primaire r1↔r2, observe la bascule OSPF sur le lien de secours r1↔r3, puis rétablit tout. Aucune configuration n'est écrite de façon permanente.

```
cd /home/floden/PPP_MCP_Telecom_Supervision
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_down
sudo ./lab/fault-injection/inject_bgp_flap.sh status
sudo ./lab/fault-injection/inject_bgp_flap.sh link_up
sudo ./lab/fault-injection/inject_bgp_flap.sh status
```

Commande	Ce que fait la commande	Résultat attendu
inject_bgp_flap.sh status	Affiche l'état nominal sans rien modifier : voisins OSPF de r1, sessions BGP de r1 et route vers le LAN serveur 192.168.3.0/24.	2 voisins Full, 2 sessions Established
inject_bgp_flap.sh link_down	Met physiquement hors service le lien primaire r1↔r2 (coupure de l'interface eth1 côté r1). Les protocoles de routage doivent réagir seuls.	Commande exécutée sans erreur
inject_bgp_flap.sh status (pendant la panne)	Re-sonde r1 pendant la coupure pour constater la dégradation et la bascule.	1 voisin OSPF, route 192.168.3.0/24 via le lien de secours 10.0.13.0/30
inject_bgp_flap.sh link_up	Rétablit le lien primaire.	Commande exécutée sans erreur
inject_bgp_flap.sh status (après guérison)	Vérifie le retour à l'état nominal : la redondance a fonctionné dans les deux sens.	2 voisins Full, 2 sessions Established

Tableau 21 — Niveau 4 : injection de panne et reconvergence.

Contrôle croisé obligatoire, à faire pendant la panne :

```
# dans l'interface graphique du serveur 2 (port 6374), onglet Tools :
#   snmp_get  ip=172.20.20.11  oid=1.3.6.1.2.1.1.1.0
```

- Résultat attendu : l'appel répond malgré la panne, en signalant l'état réel. C'est la preuve que la supervision ne s'arrête jamais, même en phase dégradée.

- Aucun collecteur ne doit planter pendant la phase dégradée : c'est le critère 9 de la grille.

D.8 Phase 5 — Niveau 5 : sécurité (5 min, terminal)

```
cd /home/floden/mcp-noc-telecom/server
CONSENT_MODE=dry_run ./venv/bin/python demo_securite.py
./venv/bin/pytest test_a6_security.py -q


cd /home/floden/noc-mcp-server && git status --short
grep -R "AuthPass2026" --exclude-dir=.git . 2>/dev/null | head


cd /home/floden/mcp-noc-telecom/server
python3 -c "import json;print([json.loads(l)['status'] for l in
open('audit.log')][-6:])"
```

Commande	**Ce que fait la commande**	**Résultat attendu**
CONSENT_MODE=dry_run … demo_securite.py	Déroule les 5 démonstrations de la couche sécurité : refus d'accès pour rôle insuffisant, autorisation selon le rôle, syslog normal, syslog piégé (injection neutralisée) et impression du journal d'audit.	DEMO 1 à 5 affichées, « FIN DE LA DEMONSTRATION », code retour 0
./venv/bin/pytest test_a6_security.py -q	Relance la suite complète d'acceptation A.6 : RBAC, JWKS, consentement, anti-passthrough, HTTP 401/200, parcours PKCE de bout en bout.	28 passed
git status --short	Affiche l'état du dépôt : vérifie que le fichier .env qui contient les secrets n'est pas suivi par git.	Aucune ligne mentionnant .env
grep -R "AuthPass2026" …	Cherche un mot de passe réel du laboratoire dans tout le dépôt, hors historique git, pour s'assurer qu'aucun secret n'est versionné.	Occurrence uniquement dans .env
python3 -c "… open('audit.log') …"	Lit les 6 dernières entrées du journal d'audit et en extrait le champ status, pour vérifier qu'aucun jeton n'y figure.	Statuts du type AUTORISE, CONSENT_DRY_RUN, REFUSE, ALERTE_INJECTION_NEUTRALISEE

Tableau 22 — Niveau 5 : sécurité, cloisonnement des secrets et audit.

D.9 Phase 6 — Contrôle global de fin de campagne (1 min, terminal)

```
for port in 6274 6374 6474; do \
  curl -s -o /dev/null -w "port $port : HTTP %{http_code}\n"
http://127.0.0.1:$port/; done
docker ps --format '{{.Names}}' | grep -c mcp-noc-lab
```

Commande	Ce que fait la commande	Résultat attendu
curl … port 6274 / 6374 / 6474	Vérifie qu'aucune des trois interfaces MCP Inspector n'est tombée pendant la campagne.	3 × HTTP 200
docker ps … | grep -c mcp-noc-lab	Vérifie que les 5 conteneurs du laboratoire sont toujours actifs en fin de test.	5

Tableau 23 — Contrôle global de fin de campagne.

D.10 Grille de résultats à remplir

À remplir au fil de l'eau : la colonne « Observé » reçoit la valeur réellement obtenue. Cette grille devient directement le tableau de synthèse du support PowerPoint.

N°	Ce qui est vérifié	Commande / action	Attendu	Observé
1	Le laboratoire est déployé	Phase 0 (docker ps)	5 nœuds UP	
2	Le routage converge	Phase 0 (vtysh OSPF / BGP)	Full + Established	
3	La suite unitaire est verte	pytest collector/tests -q	34 passed	
4	Les collecteurs répondent en réel	lab/ validate_collectors.py	9/10	
5	Le serveur expose ≥ 8 outils	Inspector, onglet Tools	9 outils (s.2 et s.3)	
6	Resources et prompts servis	Inspector, onglets Resources / Prompts	4 res. + 1 prompt (s.2)	
7	SNMPv3 authPriv fonctionne	Inspector : snmp_get sur 172.20.20.11	success	
8	La collecte CLI fonctionne	Inspector : get_bgp_peers	JSON FRR	
9	La panne est absorbée	Phase 4 (inject_bgp_flap.sh)	Bascule puis guérison	
10	La chaîne d'analyse aboutit	Phase 3 bis, serveur 3	RB-LINK-DOWN + validation humaine	

11	L'accès non autorisé est refusé	test_mcp.py / demo_securite.py	Refus RBAC	
12	L'injection est neutralisée	demo_securite.py (scénario piégé)	Neutralisation	
13	Aucun secret n'est versionné	git status + grep	.env ignoré	
14	Aucun outil ne modifie le réseau	Revue des outils dans l'IHM	Lecture seule	
15	Authentification OIDC réelle	pytest test_a6_security.py	Jeton accepté, signature falsifiée refusée	
16	OAuth 2.1 + PKCE de bout en bout	pytest test_a6_security.py (flux PKCE)	200 + S256 vérifié	
17	Consentement et anti-passthrough	pytest test_a6_security.py	28 tests verts	

Tableau 24 — Grille de résultats de la campagne (17 contrôles, colonne « Observé » à compléter).

D.11 Correspondance test → exigence → support de présentation

Réf.	**Exigence**	**Test de preuve**	**Interface**
B.2	Faisabilité en laboratoire virtuel	Phase 0	Terminal
B.4	Stack technique ciblée	Phases 1 et 2	Terminal
B.5	Minimum 8 outils MCP	Phase 3 et 3 bis	Terminal + navigateur
B.6	Lecture seule et secrets rigoureux	Phases 3 et 5	Terminal + navigateur
B.7	Analyse sans spéculation, runbook	Phase 3 bis (serveur 3)	Navigateur
B.8	Démo live incident → runbook → validation	Phases 3 bis et 4	Navigateur + terminal
A.2	Outils, resources, prompts	Phase 3 et 3 bis	Terminal + navigateur
A.3	Transports stdio / Streamable HTTP	Phase 5 (tests HTTP)	Terminal
A.4	FastMCP 3.x et décorateur @mcp.tool()	Phase 1	Terminal
A.5	Démonstration via MCP Inspector	Phase 3 bis	Navigateur
A.6	RBAC, OIDC, OAuth 2.1 + PKCE, audit	Phases 1, 5 (et parcours Keycloak)	Terminal + navigateur

Tableau 25 — Correspondance entre les tests de l'annexe D et les exigences du cahier des charges.

D.12 Points d'attention et écarts constatés

- Un redémarrage de la machine remet l'environnement dans un état partiel : les conteneurs redémarrent mais les commandes d'initialisation de containerlab (sshd, snmpd) ne se rejouent pas. Symptôme : la phase 2 tombe à 3/10 alors que la phase 0 reste verte. Correctif : sudo containerlab deploy -t lab/containerlab/topology.clab.yml (ou redeploy).
- Keycloak ne redémarre pas automatiquement (politique de redémarrage « none »). Symptôme : 10 tests sur 28 sont ignorés (skipped) et test_mcp.py s'arrête sur une erreur de connexion. Correctif : cd /home/floden/mcp-noc-telecom && docker compose up -d.
- Les interfaces MCP Inspector sont des processus, pas des services : elles doivent être relancées manuellement. Correctif : lancer chaque interface avec setsid en spécifiant un port client, un port sandbox et un port d'origine d'application distincts (6274/6275/6279, 6374/6375/6379, 6474/6476/6479).
- Le lanceur MCP Inspector est désormais installé dans le répertoire utilisateur (/home/floden/.npm/_npx/5a9d879542beca3a/...) : la variable INSP doit pointer vers ce chemin, celui de /root n'étant pas accessible sans sudo. Les interfaces doivent être lancées en compte utilisateur, jamais avec sudo, sous peine de recréer des fichiers propriétaires root.
- Le dépôt mcp-noc-ai ne contient pas de test_client_roots.py : le contrôle du serveur 3 se fait exclusivement dans l'IHM (phase 3 bis).
- Chaque redémarrage des interfaces génère de nouveaux jetons d'accès : il faut réextraire les URL avec la commande grep de la phase 3 bis.
- Les chemins des commandes sont relatifs au dépôt indiqué en tête de chaque phase : le cd est obligatoire. Exécuter une phase depuis un autre répertoire renvoie l'erreur « no such file or directory ».
- sudo est réservé à containerlab et docker. Lancer python, pip ou pytest avec sudo rend les fichiers du dépôt propriétaires de root et provoque PermissionError aux exécutions suivantes en compte utilisateur ; le correctif est sudo chown -R <utilisateur> sur les dépôts concernés.
- Le graphe containerlab est un processus séparé : si l'étape 2 indique que le port 50080 est occupé, le graphe tourne déjà et il suffit d'ouvrir http://127.0.0.1:50080 ; sinon il faut le lancer avec la commande de l'étape 2.

Fin du document.